

Abiogenesis User Guide

Engine version: 1.0.0 GTL version: 0.3.0

Contents

1. [What Abiogenesis Is](#)
 2. [Installation](#)
 3. [Your First Session](#)
 4. [The Three Commands](#)
 5. [Writing a GTL Spec](#)
 6. [Config Resolution](#)
 7. [Bootstrap Install for Other Projects](#)
 8. [The Workspace](#)
 9. [The Working Loop](#)
 10. [Traceability](#)
 11. [Understanding the Self-Hosting Spec](#)
 12. [Current Limitations](#)
-

1. What Abiogenesis Is

Abiogenesis is a GTL interpreter. GTL (Genesis Topology Language) treats software work as a typed directed graph. You define what artifacts exist, how they transform into each other, and what it means for a transformation to be complete. The engine tracks the gap between current state and done.

The six types

Type	What it is
Asset	A typed artifact — spec, code, tests, design, etc. Assets have a name and an ID format like <code>CODE-SEQ</code> .
Edge	A directed transition from one asset type to another. <code>design → code</code> is an edge.
Evaluator	A test that decides whether an edge is complete. One edge can have several evaluators.
Job	An edge paired with its evaluators. The unit of work the engine schedules.
Worker	Declares which jobs an agent or team can execute.
Package	A complete graph — all assets, edges, operators, and workers bundled together as the spec.

The three evaluator kinds

Every evaluator belongs to one of three kinds. This is the engine's core distinction:

Kind	Symbol	What it checks	Passes when
Deterministic test	F_D	Scripts, test suites, coverage checks, file scans — anything with a binary result	The command exits 0
Agent assessment	F_P	LLM or automated agent judgment — “does this output satisfy the spec?”	An agent records a passing assessment in the event log
Human approval	F_H	Explicit human sign-off	A <code>review_approved</code> event exists for this edge

The engine always runs deterministic tests first. Agent assessment only runs when all deterministic tests pass. Human approval only runs when agent assessment passes. This ordering prevents agent calls on work that has obvious deterministic failures.

Delta and convergence

Each edge has a **delta** — the count of evaluators not yet passing. `delta = 0` means that edge is converged. The workspace is converged when every edge has `delta = 0`.

The event stream

All state lives in `.ai-workspace/events/events.jsonl`, an append-only log. Assets are not stored as mutable objects — they are derived by reading the event stream. This means any past state is reconstructable by replaying the log.

The engine has three operations:

Operation	What it does
<code>gen gaps</code>	Runs deterministic evaluators across all jobs and reports residual work (delta per edge)
<code>gen iterate</code>	Selects the first unconverged job, binds context, dispatches to the appropriate evaluator kind
<code>gen start</code>	State-machine wrapper over <code>gen iterate</code> ; <code>--auto</code> loops until blocked

The workspace is converged when every evaluator on every edge reports `delta = 0`.

2. Installation

Requirements

- Python 3.11 or later
- A virtual environment is strongly recommended

Steps

```
cd /path/to/abiogenesis
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

Two packages are installed:

- **genesis** — the engine. Entry point: `gen`. Also available as `python -m genesis`.
- **gtl** — the type system. Vendored at `gtl/core.py`. Import as `from gtl.core import`
....

Verify the install

```
gen --help
```

You should see the genesis CLI help with subcommands `gaps`, `iterate`, `start`, `check-tags`, `check-req-coverage`.

3. Your First Session

Abiogenesis is self-hosting — the engine runs against its own workspace. The repo already contains a converged workspace, so this is a good first sanity check.

```
cd /path/to/abiogenesis
source .venv/bin/activate

# Check current state of the self-hosting workspace
PYTHONPATH=. gen gaps --workspace .
```

Expected output (abbreviated):

```
{
  "scope": { "package": "genesis_v1", ... },
  "jobs_considered": 5,
  "total_delta": 0,
  "converged": true,
  "gaps": [ ... ]
}
```

`total_delta: 0` and `converged: true` mean every evaluator on every edge passes. The workspace is at rest.

What `PYTHONPATH=.` does

The self-hosting spec lives at `gtl_spec/packages/genesis_core.py`. Python needs to find `gtl_spec` as a package. `PYTHONPATH=.` adds the abiogenesis root to the module search path so `from gtl_spec.packages.genesis_core import ...` resolves correctly.

When the engine is installed into another project via `gen-install.py`, the spec lives inside the target project and `PYTHONPATH` is managed by the bootstrap contract (see §7).

4. The Three Commands

All commands accept:

- `--workspace DIR` — workspace root (default: current directory)
- `--package MODULE:VAR` — override the Package to load
- `--worker MODULE:VAR` — override the Worker to load

Without `--package/--worker`, the commands read from `.genesis/genesis.yml`.

`gen gaps`

Computes residual work across the selected scope.

```
gen gaps --workspace .

# Scope to a specific feature (validates feature exists; does not filter jobs in V1)
```

```

gen gaps --workspace . --feature REQ-F-CORE

# Override package/worker ad hoc
gen gaps --workspace . \
  --package gtl_spec.packages.my_spec:my_package \
  --worker gtl_spec.packages.my_spec:my_worker

```

Output fields:

Field	Meaning
total_delta	Sum of delta across all jobs. 0 = converged.
converged	true when total_delta == 0
jobs_considered	Number of jobs evaluated
gaps[].edge	Edge name
gaps[].delta	Residual for this edge (0 = converged)
gaps[].failing	Evaluator names not yet passing
gaps[].passing	Evaluator names confirmed passing
gaps[].delta_summary	Human-readable summary line

When an edge reaches delta = 0, `gen gaps` emits an `edge_converged` event into the event log (idempotent — only one certificate per edge).

gen iterate

Selects the first unconverged job and runs exactly one bind-and-iterate pass.

```

gen iterate --workspace .

# Target a specific edge
gen iterate --workspace . --edge "design→code"

```

Output fields:

Field	Meaning
status	iterated, converged, or nothing_to_do
edge	Which edge was selected
delta_before	Delta prior to this iteration
failing_evaluators	Which evaluators were failing
events_emitted	How many events were written
prompt_words	Size of the F_P prompt (if dispatched)

What `gen iterate` does internally: 1. Runs all deterministic tests (F_D) and builds a manifest of what's passing and failing 2. If delta > 0, assembles an agent prompt with the relevant context documents 3. Walks evaluators in order: deterministic tests → agent assessment → human approval 4. Emits events from the working surface into the event log

gen start

State-machine wrapper. Without --auto, it behaves identically to gen iterate.

```
# Single iteration (equivalent to gen iterate)
gen start --workspace .

# Auto-loop: keep iterating until blocked
gen start --workspace . --auto
```

--auto stops when it encounters any condition that requires external input:

Stop reason	What happened
converged	All jobs delta = 0
stopped_by: fp_dispatch	An agent assessment was dispatched — an agent needs to act and record results
stopped_by: fh_gate	A human approval gate is waiting
stopped_by: fd_gap	A deterministic test failed — code or artifacts need fixing
stopped_by: max_iterations	Safety limit of 50 iterations reached

5. Writing a GTL Spec

A spec is a Python module that exports a Package and a Worker.

Minimal spec

```
# gtl_spec/packages/my_domain.py
from gtl.core import (
    Asset, Edge, Evaluator, Job, Operator,
    Package, Worker, F_D, F_P, F_H,
)

# — Assets
spec = Asset(name="spec", id_format="SPEC-{SEQ}")
output = Asset(name="output", id_format="OUT-{SEQ}", lineage=[spec])

# — Operator
agent = Operator("agent", F_P, "agent://claude/genesis")

# — Edge
edge = Edge(name="spec->output", source=spec, target=output, using=[agent])

# — Evaluators
eval_done = Evaluator("output_complete", F_P, "agent: output satisfies spec")
eval_tests = Evaluator(
    "tests_pass", F_D,
    "all unit tests pass",
    command="python -m pytest tests/ -q",
)

# — Job, Package, Worker
job = Job(edge=edge, evaluators=[eval_done, eval_tests])
package = Package(name="my_domain", assets=[spec, output], edges=[edge],
    operators=[agent])
worker = Worker(id="claude_code", can_execute=[job])
```

Evaluator kinds

Kind	Symbol	When to use
Deterministic test	F_D	Test suites, schema checks, tag coverage, file counts — anything with a binary pass/fail. Use <code>command=</code> to run a subprocess; exit 0 = pass.
Agent assessment	F_P	LLM or agent judgment of quality or correctness. Runs only when all deterministic tests pass.
Human approval	F_H	Explicit sign-off gate. Passes when a <code>review_approved</code> event exists for this edge in the event log.

Evaluation order

For each job, evaluators run in order: deterministic tests first, then agent assessment, then human approval. Agent assessment only runs when all deterministic tests pass. Human approval only runs when agent assessment passes. This prevents wasting agent calls on work that has obvious deterministic failures.

Multi-edge graph

A realistic spec has multiple assets and edges forming a DAG:

```
intent = Asset(name="intent", id_format="INT-{SEQ}")
req    = Asset(name="req", id_format="REQ-{SEQ}", lineage=[intent])
code   = Asset(name="code", id_format="CODE-{SEQ}", lineage=[req])
tests  = Asset(name="tests", id_format="TEST-{SEQ}", lineage=[code])

edge_i2r = Edge(name="intent→req", source=intent, target=req, using=[agent])
edge_r2c = Edge(name="req→code", source=req, target=code, using=[agent])
edge_c2t = Edge(name="code→tests", source=code, target=tests, using=[agent])

# ... evaluators and jobs per edge ...

package = Package(name="sdlc", assets=[intent, req, code, tests],
                  edges=[edge_i2r, edge_r2c, edge_c2t], operators=[agent])
worker   = Worker(id="claude_code", can_execute=[job_i2r, job_r2c, job_c2t])
```

See `gtl_spec/packages/genesis_core.py` for a complete, realistic example.

6. Config Resolution

The engine resolves `Package` and `Worker` from one of two sources:

1. **CLI flags** — `--package MODULE:VAR --worker MODULE:VAR` (highest priority)
2. **.genesis/genesis.yml** — read from `<workspace>/genesis/genesis.yml`

If neither provides a value, the command exits with error.

.genesis/genesis.yml format

```
# Genesis project config – written by gen-install.py
# Override per-invocation with: --package MODULE:VAR --worker MODULE:VAR
package: gtl_spec.packages.my_domain:package
worker:  gtl_spec.packages.my_domain:worker
```

Simple key: value pairs. Comments (#) and blank lines are ignored.

The config file is always written by `gen-install.py` on install/reinstall. It is engine metadata — safe to overwrite. The actual spec (`gtl_spec/packages/*.py`) is user data and is never overwritten by the installer.

7. Bootstrap Install for Other Projects

Use this when you want the genesis engine embedded in a project without a pip dependency on abiogenesis.

```
python /path/to/abiogenesis/builds/claude_code/code/gen-install.py \
  --target /path/to/your/project \
  --project-slug my_domain
```

What the installer does

1. Copies `builds/claude_code/code/genesis/` → `<target>/.genesis/genesis/`
2. Copies `gtl_spec/` files (`genesis_core.py`, `bootloader`, `__init__` files) → `<target>/gtl_spec/`
3. Writes `<target>/.genesis/genesis.yml` pointing to `gtl_spec.packages.my_domain:package/worker`
4. Writes `<target>/gtl_spec/packages/my_domain.py` starter spec — **only if absent** (never clobbers user edits)
5. Emits a `genesis_installed` event to `<target>/.ai-workspace/events/events.jsonl`

Running after install

```
cd /path/to/your/project
PYTHONPATH=.genesis python -m genesis gaps --workspace .
```

`PYTHONPATH=.genesis` makes the engine importable. The engine then imports the spec from the project's own `gtl_spec/packages/my_domain.py`.

Verify an existing install

```
python /path/to/gen-install.py --target /path/to/project --verify
```

Reinstall (idempotent)

```
python /path/to/gen-install.py --target /path/to/project
```

Engine files are always replaced. The starter spec is never replaced. The config file is always replaced (it is engine metadata, not user data).

8. The Workspace

The engine reads and writes `.ai-workspace/` in the workspace root.

```
.ai-workspace/
├── events/
│   └── events.jsonl      ← append-only event log (the ground truth)
├── features/
│   ├── active/          ← feature vectors currently in-progress (YAML)
│   └── completed/        ← converged feature vectors (YAML)
├── reviews/
│   └── pending/          ← human review proposals
└── comments/
    └── <agent>/          ← per-agent discussion layer
```

The event log

`events.jsonl` is the substrate. Every event is a JSON line:

```
{"event_type": "edge_converged", "event_time": "2026-03-15T...", "data":  
  {"edge": "design→code", "target": "code", "delta": 0}}
```

Assets are projections over the event stream — not stored directly. When you ask “has this edge converged?”, the engine scans the event log for `edge_converged` events.

The event log is append-only. Never edit or delete lines.

Feature vectors

Feature YAML files live in `.ai-workspace/features/active/` or `completed/`. A file named `REQ-F-CORE-001.yml` creates a known feature ID. `gen gaps --feature REQ-F-CORE-001` validates that this ID exists (V1 — it does not yet narrow which jobs run).

9. The Working Loop

The standard cycle when doing active development:

```
# 1. See what's left  
gen gaps --workspace .  
  
# 2. If work is pending, run one iteration  
gen iterate --workspace .  
# (or: gen start --workspace . --auto to loop until blocked)  
  
# 3. Read what happened  
tail -5 .ai-workspace/events/events.jsonl  
  
# 4. Respond to what the engine reported  
#   - fd_gap_found → fix the deterministic failure, then go to step 1  
#   - fp_dispatched → do the agent work, record the assessment, then go to  
#     step 1  
#   - fh_gate_pending → provide human approval (emit review_approved event),  
#     then go to step 1  
  
# 5. Confirm delta dropped  
gen gaps --workspace .
```

The workspace is done when `gen gaps` reports `converged: true` and `total_delta: 0`.

10. Traceability

The engine ships two subcommands for verifying code-spec traceability.

Check implementation tags

Every non-`__init__` Python file in the engine should carry at least one `# Implements: REQ-*` tag:

```
gen check-tags --type implements --path builds/claude_code/code/
```

Output is JSON: `{"passes": true, "untagged_count": 0, "untagged": []}` on success.

Check test tags

Every non-`__init__` test file should carry at least one `# Validates: REQ-*` tag:

```
gen check-tags --type validates --path builds/claude_code/tests/
```

Check requirement coverage

Every REQ key in the spec package should appear in at least one feature vector:


```
gen check-req-coverage \
  --package gtl_spec.packages.genesis_core:genesis_v1 \
  --features .ai-workspace/features/
```

Also accepts `--spec path/to/spec.md` for a grep-based scan of a markdown spec file (legacy path).

11. Understanding the Self-Hosting Spec

`gtl_spec/packages/genesis_core.py` is the V1 specification written as GTL. It defines:

Assets (6):

Asset	id_format	Meaning
intent	INT- {SEQ}	The goal to build
requirements	REQ- {SEQ}	Tech-agnostic requirements
feature_decomp	FD- {SEQ}	Feature breakdown
design	DES- {SEQ}	Architecture and ADRs
code	CODE- {SEQ}	Implementation
unit_tests	TEST- {SEQ}	Test suite

Edges (5):

Edge	Evaluators	Kind
intent→requirements	intent_approved	Human approval
requirements→feature_decomp	req_coverage, feat_approved	Deterministic test, Human approval
feature_decomp→design	design_complete, design_approved	Agent assessment, Human approval
design→code	impl_tags, six_modules, code_complete	Deterministic test, Deterministic test, Agent assessment
code→unit_tests	tests_pass, validates_tags, test_complete	Deterministic test, Deterministic test, Agent assessment

Worker: `worker_claude_code` — can execute all 5 jobs.

The self-hosting workspace is currently converged (`total_delta: 0`). The event log in `.ai-workspace/events/events.jsonl` contains the certificates.

12. Current Limitations

V1 scoping

--feature REQ-F-CORE-001 validates that the feature ID exists in `.ai-workspace/features/` but does not yet narrow which jobs run. All jobs run regardless of feature scope. Per-job feature routing is V2.

Agent dispatch is asynchronous

When an agent assessment evaluator (F_P) fires, `gen_iterate` emits an `fp_dispatched` event and stops. The agent (Claude, Codex, etc.) reads this event, does the work, and records the result externally. The engine does not invoke the agent directly — the agent invokes the engine. This is intentional: it keeps the engine pure and the agent interaction explicit.

Human approval requires a manual event

To clear a human approval gate (F_H), emit a `review_approved` event into the event log manually or via your agent:

```
{"event_type": "review_approved", "event_time": "...", "data": {"edge":  
    "design-code", "actor": "human"}}
```

Once this event exists, `bind_fd` will see the approval and the F_H evaluator will pass on the next `gen_gaps / gen_iterate` call.

Single worker

V1 has one worker (`claude_code`). Multi-tenant scheduling with conflict detection is deferred to V2. The `schedule()` function and `Worker.conflicts_with()` are implemented but only exercised with a single worker in V1.

Source layout

The engine source is at `builds/claude_code/code/genesis/` — not `src/genesis/`. The `builds/` prefix reflects the abiogenesis self-hosting structure: the engine is itself a build artifact that the spec describes. A conventional `src/` layout is deferred until the packaging migration is complete.